THERS - App móvil Android

Cómo cambiar de máquina sin problemas

Caso hipotético: se instala todo el entorno Android en esta computadora (la que hoy no lo tiene) y se quiere seguir trabajando también en la computadora anterior (la que ya compiló y probó la app en el moto z3), sin que una estorbe a la otra.

> **Idea central.** El código viaja por git. El entorno Android (SDK, JDK, variables, la carpeta `mobile/android/`, el `.env`) NO viaja: vive en cada máquina y no se comparte. Por eso instalar todo aquí no altera la otra computadora. Lo único que puede dar guerra es el teléfono (firma de la app) y los pasos de sincronizar el código.

Base documental: este documento resume `docs/mobile/ANDROID_SETUP.md`, `docs/mobile/VALIDATION.md` y `mobile/.env.example`. Si hay diferencia, manda `ANDROID_SETUP.md` (es el que se verificó en una máquina real). Los tiempos de este documento son estimaciones, no mediciones.
Contenido: 1 Qué es de cada máquina - 2 Instalar el entorno en esta computadora - 3 Primera compilación - 4 Pasar el trabajo de una máquina a la otra - 5 Volver a la máquina anterior - 6 El teléfono - 7 Backend y base de datos - 8 Problemas frecuentes - 9 Lista de verificación.

1. Qué es de cada máquina y qué viaja por git

Elemento	Dónde vive	¿Viaja por git?
Código de `mobile/`, `package.json`, `package-lock.json`, `app.json`	Repositorio	Sí (cuando se haga commit y push)
JDK 17, Android Studio, SDK, NDK, CMake	Carpetas del sistema o del usuario	No
Variables `JAVA_HOME`, `ANDROID_HOME`, `PATH`	Entorno de cada máquina	No
`mobile/node_modules/`	Cada máquina (se regenera)	No (ignorado)
`mobile/android/` (proyecto nativo generado)	Cada máquina	No (ignorado)
`mobile/.env` (URL del backend)	Cada máquina	No (ignorado; nunca se sube)
Regla de firewall del puerto 5000	Cada máquina	No
Unidad virtual `M:` (`subst`)	Cada máquina; se pierde al reiniciar	No
Llave de depuración (`debug.keystore`)	Cada máquina (en `%USERPROFILE%\.android`)	No
Base de datos local (PostgreSQL en Docker)	Cada máquina	No

Consecuencia: instalar el entorno en una máquina no cambia nada en la otra. Los archivos generados (`android/`, `.env`, `node_modules/`) están en el `.gitignore` de `mobile/`, así que tampoco ensucian `git status`.

2. Instalar el entorno Android en esta computadora

Estimado: más de una hora entre descargas e instalación, sin contar la primera compilación. Ocupa varios GB (la guía anotó 41 GB libres después de SDK 36 + NDK, y faltan unos 1,5 GB de `node_modules` y 3 a 5 GB de cachés de Gradle). Revisa el espacio libre antes.

Versiones fijadas por el proyecto (Expo SDK 57)

Pieza	Versión	Nota
Node / npm	Node 20 o superior	Aquí hay Node v24.14.1: sirve
JDK	17 (Temurin)	NO usar el JDK 25 que trae Android Studio: su `keytool` falla en español y no está verificado con este proyecto
Android SDK	`platforms/android-36`	compileSdk y targetSdk 36; minSdk 24
NDK	`27.1.12297006`	exigido por React Native 0.86.3
Build-Tools / CMake	35.0.0 y 3.22.1 (además de 36/37)	Gradle los descarga solo la primera vez
Gradle / Kotlin	9.3.1 / 2.1.20	Los trae el proyecto; no se instalan a mano

Paso 1 - Android Studio

Instala Android Studio (trae el SDK y `adb`). Ábrelo una vez para que descargue el SDK en `%LOCALAPPDATA%\Android\Sdk`.

Paso 2 - JDK 17 (en PowerShell; puede pedir permisos de administrador)

```
winget install --id EclipseAdoptium.Temurin.17.JDK --accept-package-agreements --accept-source-agreements
Get-ChildItem "C:\Program Files\Eclipse Adoptium"      # anota la carpeta jdk-17... exacta
```

Paso 3 - Variables de entorno (ajusta la ruta exacta del JDK que anotaste)

```
setx JAVA_HOME "C:\Program Files\Eclipse Adoptium\jdk-17.0.20.101-hotspot"
setx ANDROID_HOME "$env:LOCALAPPDATA\Android\Sdk"
setx PATH "$env:JAVA_HOME\bin;$env:LOCALAPPDATA\Android\Sdk\platform-tools;$env:PATH"
```

> **Cierra y abre una terminal nueva** después de esto. Una terminal abierta antes sigue viendo el Java viejo. Comprueba con `java -version` (debe decir 17) y `adb version`.

Paso 4 - Paquetes del SDK (la sintaxis importa)

El comando antiguo `sdkmanager` con nombres que llevan punto y coma falla en silencio y sale con código 0. Usa el CLI nuevo con barras:

```
$android = "$env:LOCALAPPDATA\Android\Sdk\cmdline-tools\latest\bin\android.exe"
& $android sdk install "platforms/android-36" "ndk/27.1.12297006"
& $android sdk list                       # verifica lo instalado
Get-ChildItem "$env:LOCALAPPDATA\Android\Sdk\platforms", "$env:LOCALAPPDATA\Android\Sdk\ndk"
```

Verifica en disco que existan `android-36` y `27.1.12297006`, no solo el código de salida. Si una ejecución se interrumpe y da "Acceso denegado (os error 5)", borra los residuos: `%USERPROFILE%\.android\cli\bundles\*.extract` y el archivo `lock` de esa carpeta.

Paso 5 - Firewall del puerto 5000 (terminal de administrador)

Sin esta regla, el teléfono no alcanza el backend por Wi-Fi. Acotada a redes privadas:

```
New-NetFirewallRule -DisplayName "THERS backend dev 5000" -Direction Inbound `
  -LocalPort 5000 -Protocol TCP -Action Allow -Profile Private
```

Si tu Wi-Fi está marcado como red pública, esa regla no se aplicaría. Lo preferible es cambiar la red a Privada; la alternativa es `-Profile Private,Public`, que abre el puerto también en redes públicas.

Paso 6 - Depuración USB en el teléfono

Activa las opciones de desarrollador (toca varias veces el número de compilación), enciende la depuración USB, conecta el cable y acepta el aviso de confianza en la pantalla. Comprueba:

```
adb devices        # el moto z3 debe aparecer como 'device', no como 'unauthorized'
```

3. Primera compilación en esta computadora

Primero necesitas el código. Mientras mis cambios sigan sin commit, vive solo en esta máquina, así que en esta no hay nada que traer. En la otra máquina sí: ve a la sección 4.

a) Dependencias

```
cd <repo>\mobile
npm ci --legacy-peer-deps
```

El flag es necesario: sin él `npm ci` falla por un conflicto de pares previo entre `expo` y `react-dom`. No modifica `package.json` ni el lockfile.

b) Variable de entorno

Copia `mobile/.env.example` a `mobile/.env` y pon la IP de ESTA computadora en la red (`ipconfig`, adaptador Wi-Fi; no la del adaptador vEthernet de WSL):

```
EXPO_PUBLIC_API_URL=http://<IP-de-esta-maquina>:5000/api
```

> Esa variable queda **embebida en la app**. Nunca pongas ahí secretos (claves JWT, de base de datos, de correo ni de S3). La IP cambia con la red: si algún día no conecta, empieza por revisarla.

c) Unidad corta para evitar rutas largas de Windows

La compilación nativa puede fallar con "Filename longer than 260 characters". La solución probada es una unidad virtual a la **raíz del repositorio** (no a `mobile/`) y compilar desde ahí:

```
subst M: "C:\ruta\al\repositorio\THERS_REDSOCIAL"
cd M:\mobile
npm run android
```

`subst` se pierde al reiniciar: hay que repetirlo. Mapear `M:` directo a `mobile/` falla. Los avisos de Kotlin sobre "different roots" no impiden compilar.

d) Qué esperar

`npm run android` compila, instala la "development build" en el teléfono y deja Metro en el puerto 8081 con un `adb reverse`. La primera vez puede tardar de 15 a 30 minutos o más (la guía anotó que una build larga rozó el límite de 30 minutos de una sesión en segundo plano). Si se corta, vuelve a lanzarla: retoma desde la caché. Si Metro se cierra, la app instalada no puede cargar su JavaScript: arráncalo con `npm start` sin recompilar.

> **Esta primera compilación es obligatoria** aunque la otra máquina ya haya compilado antes. Las ramas recientes agregaron `expo-font` y `react-native-svg`, que son módulos nativos: una app instalada con la versión anterior falla al abrir.

4. Pasar el trabajo de una máquina a la otra

Hoy los cambios móviles (tema oscuro y claro, tipografía, iconos, T-Rayo, etc.) están en la rama `feature/mobile-obsidian-shell`, **sin commit**, solo en la máquina donde se hicieron. Para llevarlos a la otra:

En la máquina de origen

```
git status                                    # revisa qué se va a subir
git add mobile/ docs/
git commit -m "feat(mobile): tema oscuro y claro, tipografia unica e iconos"
git push -u origin feature/mobile-obsidian-shell
```

> Según `CLAUDE.md` §8.7, Claude solo hace commit o push cuando se le pide en ese turno. Pídeselo explícitamente. Antes de `git add` revisa que no entren archivos sueltos que no son del móvil (en el repo hay sin versionar un `.docx`, un PDF y una imagen) ni ningún `.env`.

Alternativa sin git (no recomendada)

Copiar la carpeta `mobile/` con un USB o la nube. Excluye `node_modules/`, `android/` y `.env`: se regeneran y el `.env` es de cada máquina. Pierdes el historial y es fácil dejar fuera algún archivo

nuevo.

5. Volver a la máquina anterior

Esa máquina ya tiene el entorno. Solo hay que poner el código al día y recompilar:

```
git fetch
git switch feature/mobile-obsidian-shell
cd M:\mobile                       # recrea antes: subst M: <ruta al repo>
npm ci --legacy-peer-deps          # trae expo-font, react-native-svg y la fuente
npm run android                    # recompila: hay módulos nativos nuevos
```

Si ves…	Qué significa y qué hacer
`INSTALL_FAILED_UPDATE_INCOMPATIBLE`	El teléfono tiene la app firmada por la otra máquina. Desinstala THERS del teléfono y vuelve a instalar (ver sección 6).
La app abre y se cierra, o error rojo de módulo nativo	Falta recompilar: `expo-font` / `react-native-svg` no están en la app instalada. Ejecuta `npm run android`.
`npm ci` falla con ERESOLVE	Olvidaste `--legacy-peer-deps`.
Sin conexión con el backend	La IP del `.env` de esa máquina cambió, o falta la regla de firewall.

6. El teléfono: lo único que se comparte entre las dos máquinas

Firma de la app

Cada máquina firma la build de desarrollo con su propia llave de depuración. Android no deja actualizar una app con una firma distinta. Cuando cambies de máquina para instalar:

```
adb uninstall com.thers.app        # o desinstálala a mano desde el teléfono
npm run android                    # instala la build firmada por esta máquina
```

Desinstalar borra los datos de la app en el teléfono (la sesión guardada). Basta con volver a iniciar sesión.

Metro y el puerto 8081

`npm run android` hace el `adb reverse tcp:8081`, así que Metro llega por el cable USB sin tocar el firewall. Sin cable, el teléfono no encuentra Metro. Si dos máquinas están conectadas al mismo teléfono a la vez, `adb` puede confundirse: conecta el teléfono a una sola a la vez.

Google Sign-In (solo si lo vas a probar)

El backend acepta una sola audiencia (`GOOGLE_CLIENT_ID`), y en Google Cloud Console debe registrarse un cliente OAuth de tipo Android con el paquete `com.thers.app` y el SHA-1 de la llave de depuración **de cada máquina**. Si registras solo la de una, el botón de Google falla en la otra; el inicio de sesión con correo y contraseña no se ve afectado. Se obtiene con:

```
& "C:\Program Files\Android\Android Studio\jbr\bin\keytool.exe" `
  "-J-Duser.language=en" "-J-Duser.country=US" `
  -list -v -keystore "$env:USERPROFILE\.android\debug.keystore" `
  -alias androiddebugkey -storepass android -keypass android
```

El `-J-Duser...` es el rodeo del error de idioma del `keytool`. La llave de depuración y su contraseña son públicas por diseño de Android; la llave de *release* es secreta y nunca entra al repositorio.

7. Backend y base de datos en cada máquina

El teléfono habla con el backend que corra en la máquina cuya IP pusiste en el `.env`. Cada máquina tiene su propia base de datos local, así que las cuentas y publicaciones creadas en una no aparecen en la otra.

```
cd <repo>
docker compose up -d                          # PostgreSQL 16 (puerto 5432, o POSTGRES_PORT)
cd backend
venv\Scripts\activate
pip install -r requirements-dev.txt
set FLASK_APP=run.py                          # PowerShell: $env:FLASK_APP = "run.py"
python -m flask db upgrade
python run.py                                 # escucha en 0.0.0.0:5000
```

Si en esa máquina ya hay un PostgreSQL nativo en el 5432, fija `POSTGRES_PORT` a otro puerto y ajusta `DATABASE_URL` con ese mismo puerto (ver `CLAUDE.md` §11). Las peticiones nativas de la app no pasan por CORS: si algo falla en el teléfono, revisa firewall, IP y el encabezado `Authorization`, no `CORS`.

8. Problemas frecuentes

Síntoma	Causa probable	Qué hacer
`java -version` muestra 8 o 25	Terminal abierta antes de instalar, o PATH con otro JDK delante	Abrir terminal nueva; revisar que `JAVA_HOME` apunte al 17
`adb` no se reconoce	`platform-tools` fuera del PATH	Repetir el Paso 3 y abrir terminal nueva
`adb devices` dice *unauthorized*	No aceptaste el aviso en el teléfono	Desbloquear el teléfono y aceptar; revocar autorizaciones USB si no sale
Error *Filename longer than 260 characters*	Rutas largas en la parte C++	Compilar desde `M:\mobile` (sección 3c)
Couldn't find package.json up from path	`M:` mapeada a `mobile/`	Mapear `M:` a la raíz del repo
Package platforms not found pero código 0	Sintaxis con punto y coma en el SDK Manager	Usar `android sdk install "platforms/android-36"` y verificar en disco
Build detenida a mitad	Límite de tiempo o poca memoria	Relanzar: retoma desde la caché de Gradle
Expo reescribió `mobile/tsconfig.json`	Pasa al arrancar	Revisar el diff antes de hacer commit
La app no carga el JavaScript	Metro cerrado	`npm start` sin recompilar
`git status` avisa *Filename too long* sobre `.cxx`	Caché ignorada con rutas largas	Es ruido; ignorar

9. Lista de verificación (marca cada paso)

Paso	Esta máquina	La anterior
JDK 17, `JAVA_HOME`, `ANDROID_HOME`, `PATH` (terminal nueva)	[ ]	ya está

Paso	Esta máquina	La anterior
SDK 36 y NDK 27.1.12297006 verificados en disco	[ ]	ya está
Regla de firewall del puerto 5000	[ ]	ya está
Rama `feature/mobile-obsidian-shell` subida y traída	[ ]	[ ]
`npm ci --legacy-peer-deps` en `mobile/`	[ ]	[ ]
`mobile/.env` con la IP de esa máquina	[ ]	[ ]
`subst M:` hecho tras el último reinicio	[ ]	[ ]
App anterior desinstalada del teléfono (si cambia la firma)	[ ]	[ ]
`npm run android` compila e instala	[ ]	[ ]
Backend y PostgreSQL arriba en la máquina cuya IP está en el .env	[ ]	[ ]
Inicio de sesión funciona en el teléfono	[ ]	[ ]
Se ven los modos oscuro y claro (cambiar el tema del sistema)	[ ]	[ ]
Capturas enviadas para revisar el diseño	[ ]	[ ]

> Lo que NO está verificado: el modo claro, la tipografía Space Grotesk, el contador de Mensajes y el resto de los cambios recientes se comprobaron solo con typecheck, pruebas y el bundle de Android, nunca en un teléfono. Esta guía es justamente para poder verlos. Además, la app nunca se ha probado en el Samsung A16 5G, solo en el moto z3.